

Proiect 2 - POO

Mocanu Ștefan

1 Timeline

- Deadline – joi 21 mai (23:59) - ultimul commit (să am timp să mă uit pe proiecte)
- Prezentare – luni 25 mai (pentru excepții accept să-mi prezentați mai devreme, dar anunțați pe mail stefan.mocanu1@s.unibuc.ro să discutăm situația)

2 Variante de Proiect

Puteți alege una dintre următoarele două direcții:

Varianta 1: Sistem Aplicat (Recomandat): Implementarea unui proiect complex care utilizează biblioteci externe (ex: *SFML/OpenGL* pentru grafică, *nlohmann/json* pentru parsing, *Crow/Dragon* pentru API-uri HTTP, *SQLite/PostgreSQL* pentru baze de date).

Varianta 2: Modelare Abstractă (Maxim nota 7): Similar cu Proiectul 1, o modelare a unui sistem din viața reală, dar care respectă toate cerințele tehnice de mai jos (inclusiv Templates și Design Patterns).

3 Cerințe Tehnice Obligatorii

Indiferent de varianta aleasă, proiectul trebuie să demonstreze următoarele:

- ◇ **Organizarea Codului:** Separarea interfeței de implementare (`.h` / `.hpp` și `.cpp`).
- ◇ **Paradigma OOP Pură:** Utilizarea exclusivă a claselor. Nu se admit variabile globale sau funcții globale (cu excepția funcției `main`).
- ◇ **Fundamente Avansate:** Utilizarea corectă a moștenirii, polimorfismului, membrilor și funcțiilor `static`.
- ◇ **Genericitate (Templates):** Implementarea a cel puțin unei clase sau funcții template (șablon) care să aducă valoare logică sistemului.
- ◇ **Design Patterns (Șabloane de Proiectare):** Implementarea a cel puțin două șabloane de proiectare diferite de `Singleton` (ex: *Factory*, *Strategy*, *Observer*, *Command*, *Decorator*, *State*).
- ◇ **Versionare:** Utilizarea **GitHub** pentru istoricul commit-urilor.

4 Sugestii de Tematici

Dacă sunteți în căutarea inspirației, iată câteva direcții cu un grad ridicat de complexitate:

- **Dezvoltare Jocuri:** Un motor de joc simplu sau un joc complet folosind *SFML* sau *OpenGL*.
- **Custom Command-Line Shell (CLI Shell):** Implementarea unei console proprii care permite navigarea în sistemul de fișiere și execuția unor comenzi interne complexe. (Nu necesită librării externe)
- **Networking & Web:** Un API REST care gestionează o bază de date de resurse (ex: un sistem de rezervări).
- **Compilatoare și Parsere:** Un interpretor pentru un limbaj de programare custom.
- **Metaprogramare (Hardcore):** Deoarece Template-urile în C++ sunt *Turing-complete*, puteți scrie un parser care citește un limbaj simplu (cu IF-uri și loop-uri) și îl rescrie sub formă de template-uri care să execute codul la momentul compilării. La primul laborator după vacanță discutăm despre templates, cine ar fi interesat mă poate contacta pentru ajutor.

5 Evaluare și Flexibilitate

Sunt deschis la negocieri punctuale în privința cerințelor tehnice dacă proiectul compensează prin complexitate excepțională într-o anumită zonă (ex: o implementare foarte avansată de grafică poate scuti utilizarea template-urilor).

6 Înregistrare și Administrare

Fiecare student trebuie să își înregistreze tema aleasă și link-ul către repository-ul de GitHub în documentul centralizator de mai jos.

6.1 Link Înregistrare

CLICK AICI PENTRU COMPLETARE TEMA

6.2 Continuarea Proiectului 1

Dacă ați ales să păstrați tematica de la primul proiect și doriți să o extindeți, să specificați acest lucru în coloana corespunzătoare temei din tabel prin mențiunea:

"bis proiect1"